



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Chap 4. Process Models

❖ 0. What is a Prescriptive Process Model?

❖ 1. The Waterfall Model & the V Model

❖ 2. The Incremental Model

❖ 3. Evolutionary Models

- Prototyping
- The Spiral

❖ 4. Other Process Models

❖ 5. The Unified Process (UP)

- 规范的软件过程模型 ☆
 - 框架活动
 - 软件工程活动
 - 任务
 - 产品
 - 质量保证
 - 变更控制

什么是规范过程模型？

What is a Prescriptive Process Model?

❖ 又被称为： 软件工程经验模型， 软件开发模型 等等。

❖ The **Prescriptive process models** are the **accumulation of experience** in the process of software development. 规范过程模型是在软件开发过程中经验的积累。

❖ It determines the **structural framework** of the whole process, activities and tasks of software development. 确定了整个过程、活动和任务的结构框架

❖ According to different adaptability, each model has different emphasis on framework activities.

根据不同的适应性，
每种模型在框架活动上有不同的侧重点。

The Framework Activities of a Prescriptive Process Model

规范过程模型的框架活动

❖ Communication

- Application Requirements determination, Software Requirements specification

❖ Planning

- Planning, estimation, scheduling

❖ Modeling

- Architectural design, Detailed design

❖ Construction:

- Implementation, Integration, Testing

❖ Deployment

交流
应用需求确定，软件
需求规格说明
计划
规划，估算，调度
建模
架构设计，详细设计
构建
实施，集成，测试
部署

The Waterfall Model

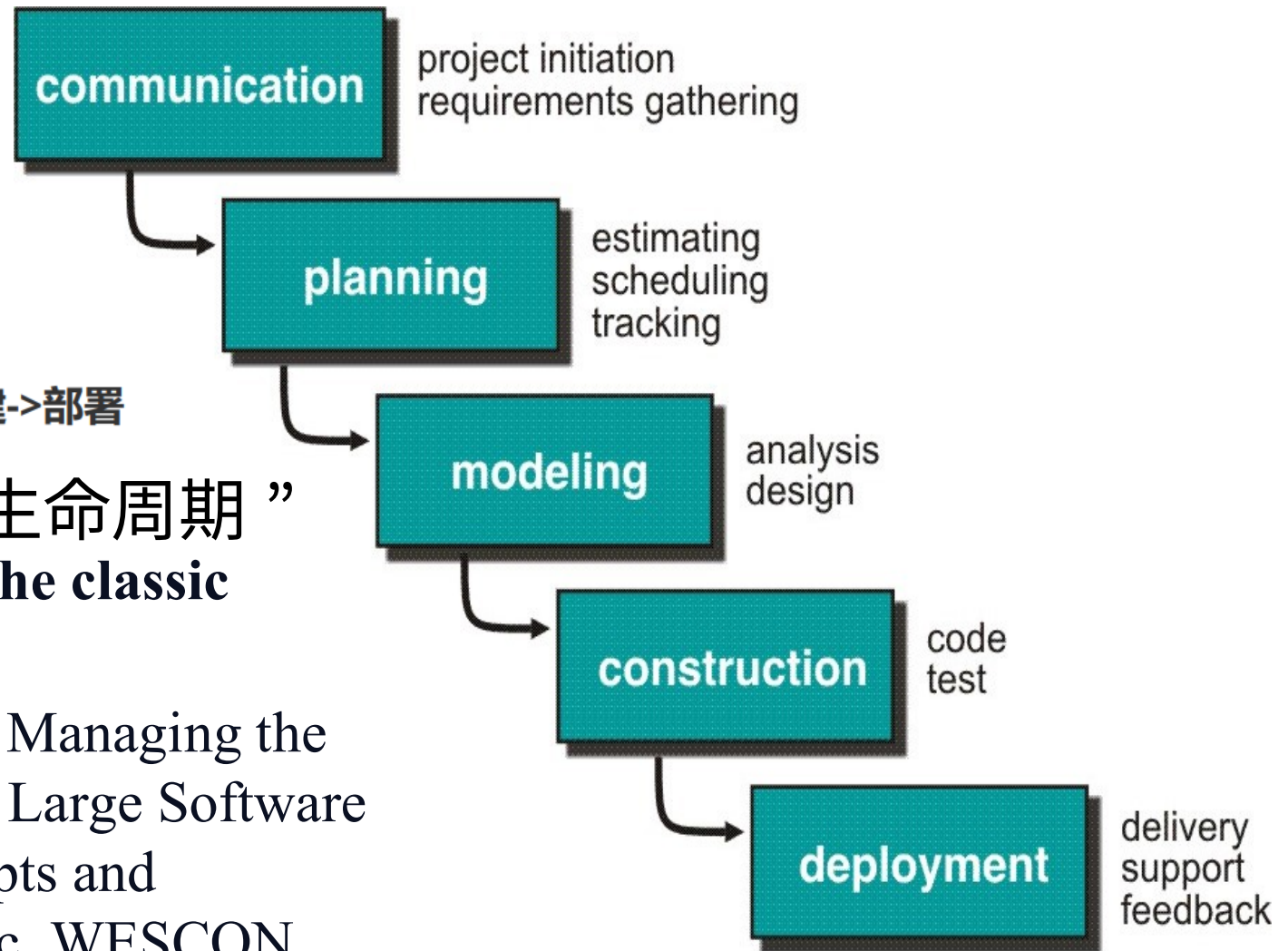
1. 瀑布模型 ☆ ☆

- 沟通->策划->建模->构建->部署

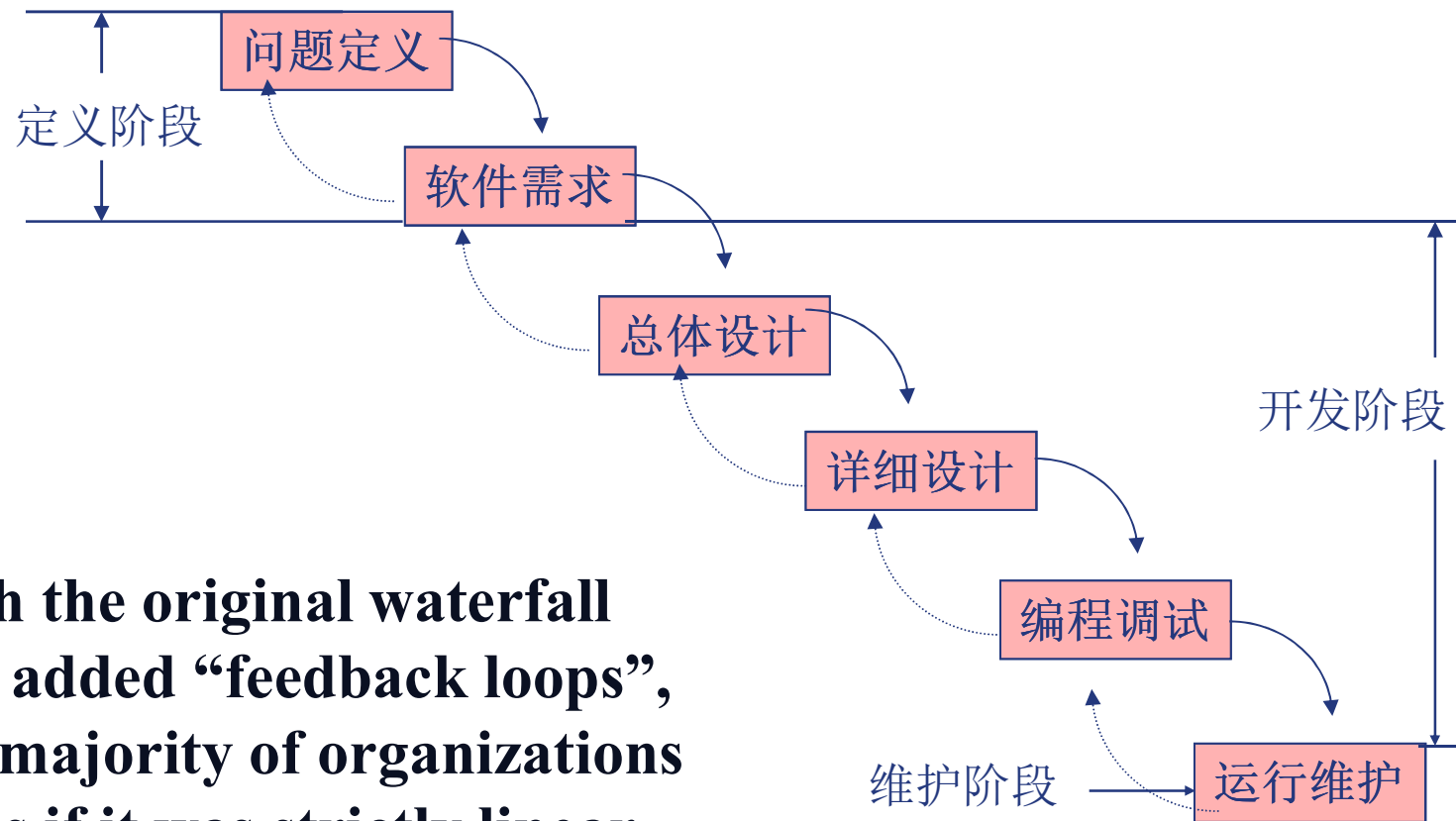
也被称为“经典生命周期”

- ❖ Also called “The classic lifecycle”
- ❖ Winston Royce, Managing the Development of Large Software Systems: Concepts and Techniques, Proc. WESCON, 1970

温斯顿·罗伊斯，《大型软件系统开发管理：概念与技术》，WESCON会议，1970年



The Waterfall Model



- ❖ Although the original waterfall model is added “feedback loops”, the vast majority of organizations treat it as if it was strictly linear.

尽管最初的瀑布模型添加了“反馈环路”，但绝大多数组织仍将其视为严格的线性模型。

The Waterfall Model

❖ Characteristics of the waterfall model

- **Accept the work product (work product) of the activity from the previous activity as input;**
- **Implement the activity on the basis of this input;**
- **The work product of this activity will be transmitted to the next activity;**
- **Before the transmission, the work product will be reviewed and confirmed.**

■ 特点:

- 从上一项活动中接受该项活动的工作成果(工作产品), 作为输入
- 利用这一输入实施该项活动应完成的内容
- 给出该项活动的工作成果, 作为输出传给下一项活动
- 对该项活动实施的工作进行评审。若其工作得到确认, 则继续下一项活动

The Waterfall Model

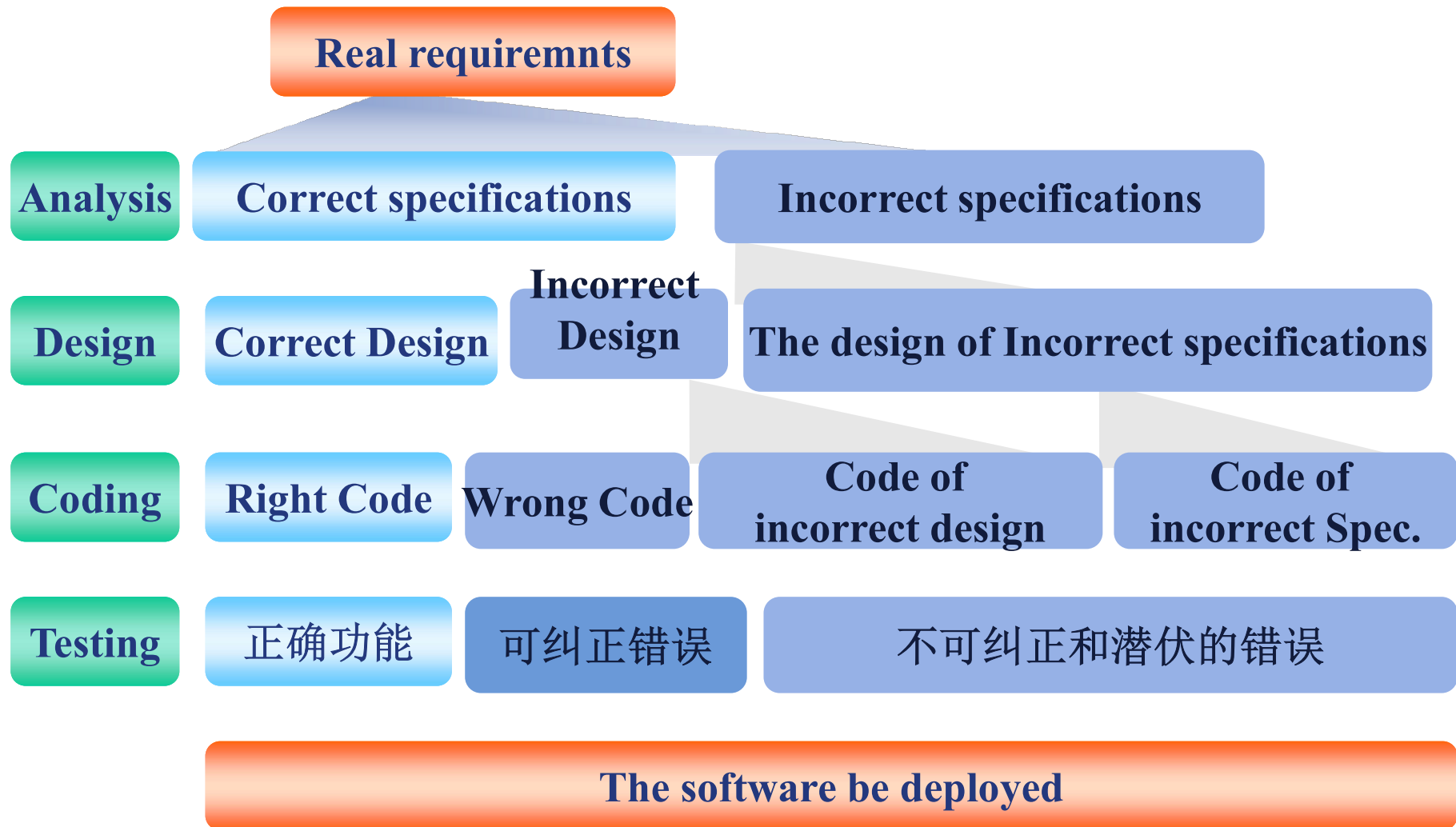
❖ 瀑布模型的**优点**:

- 强调开发的阶段性;
- 强调早期计划及需求调查;
- 强调产品测试。

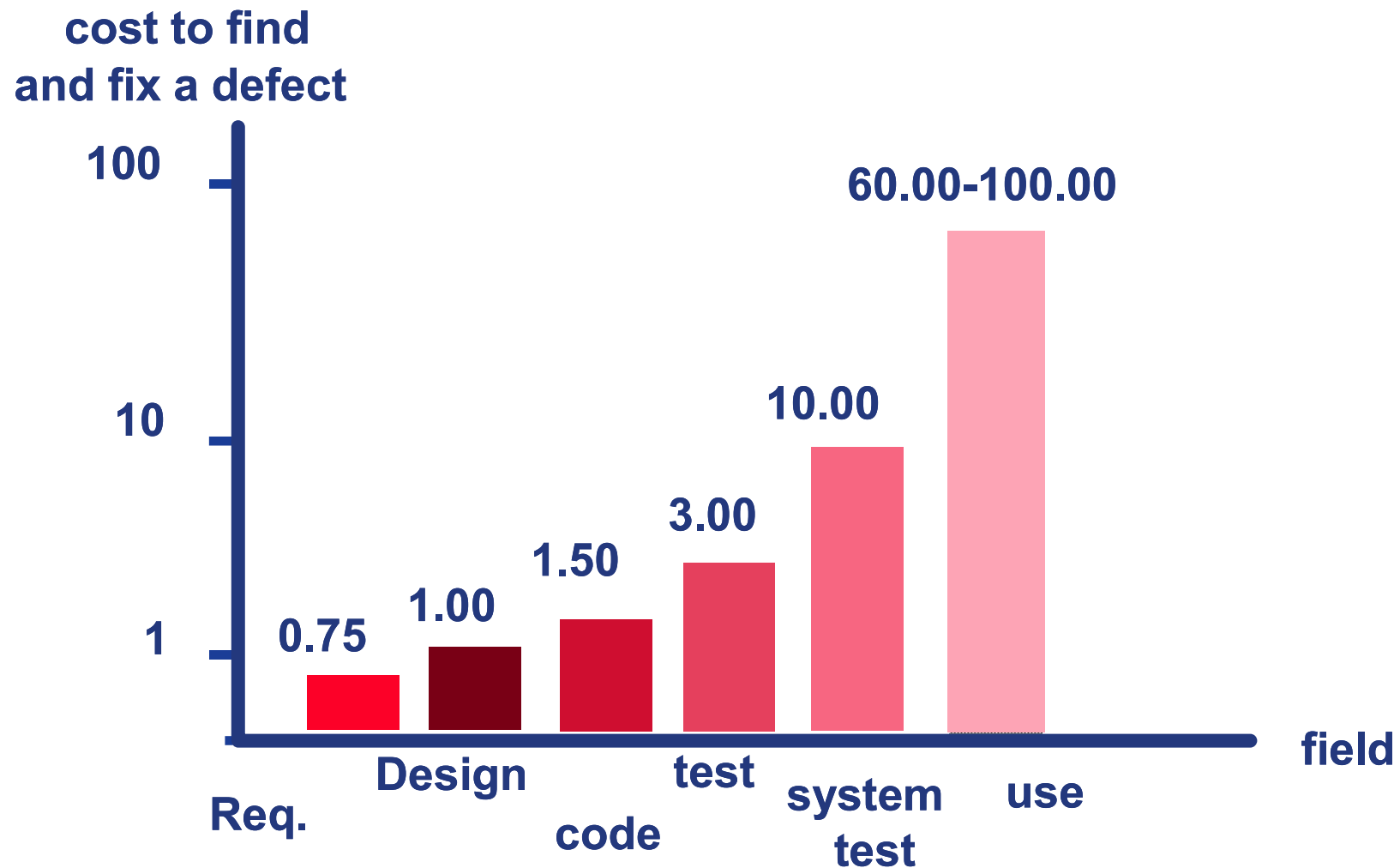
❖ 瀑布模型的**缺点**:

- 瀑布模型过于依赖早期进行的唯一一次需求调查, 不能适应需求的变化
- 瀑布模型是单一流程, 开发中的经验教训不能反馈应用于本产品的过程。

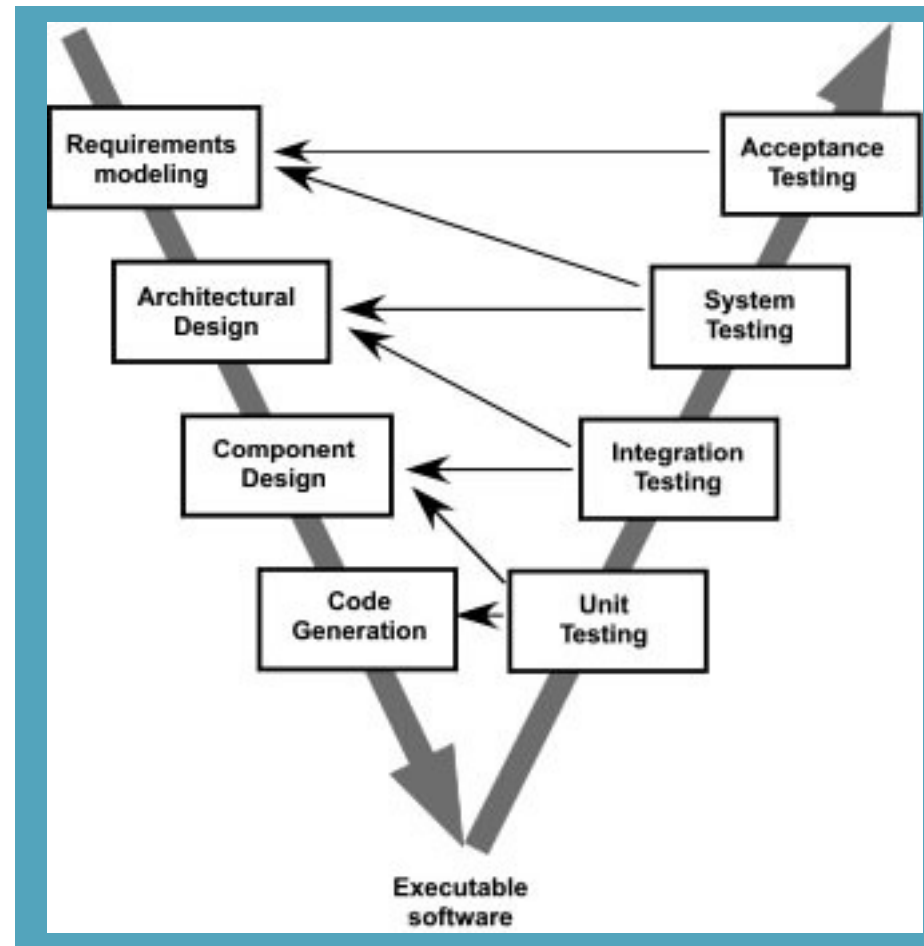
瀑布模型造成软件错误的积累和放大效应



瀑布模型造成软件错误的积累和放大效应



The V-Model (A variant of the waterfall model)

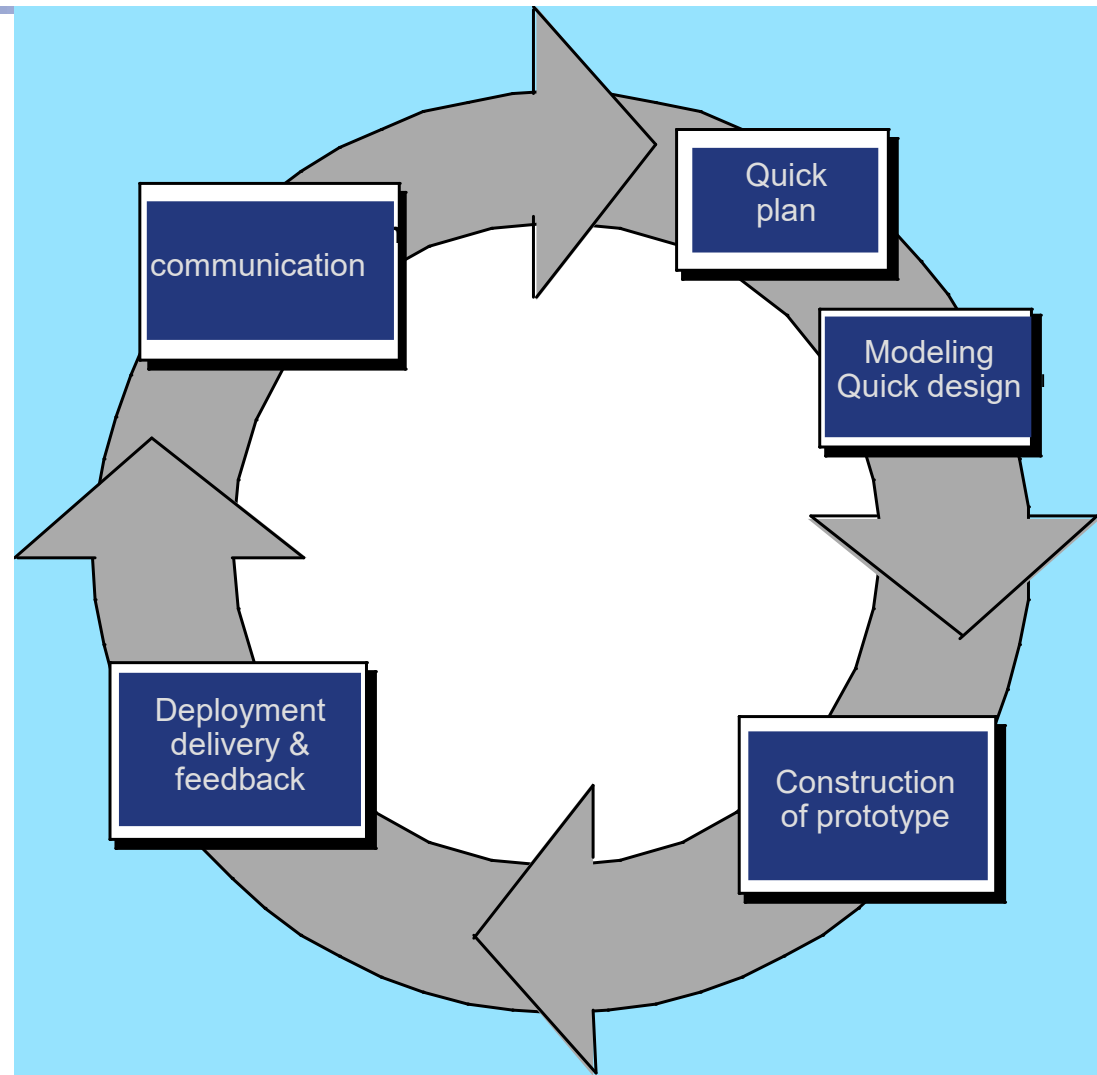


The Waterfall Model

- 关于瀑布模型的思考：
 - 我们的日常生活中，有哪些活动是符合瀑布模型的？
 - 你们的实训过程是瀑布模型吗？
 - 为什么我们的小型开发团队很难坚持做到瀑布模型？客观因素是什么？
 - 瀑布模型的现实实用价值在哪里？

Evolutionary Models

- **Business requirements often change**
- **Tight market deadlines, only allow a limited version**
- **Only core requirements are defined**

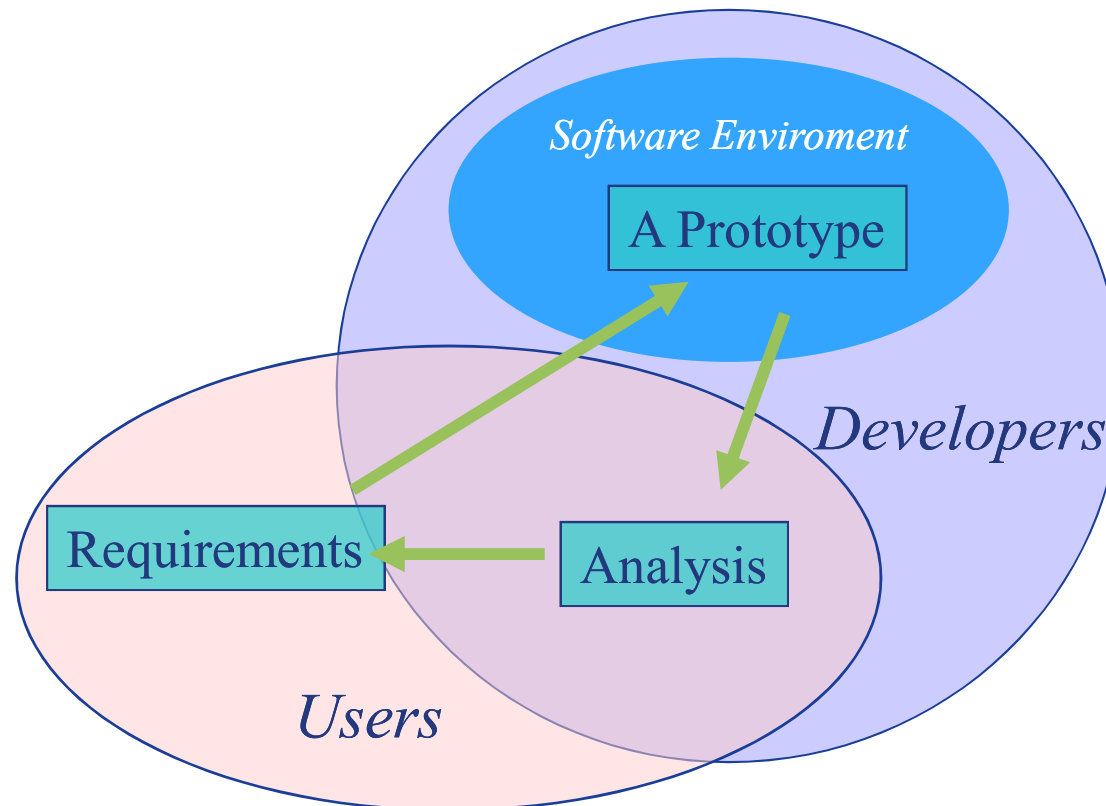


■ 适用范围

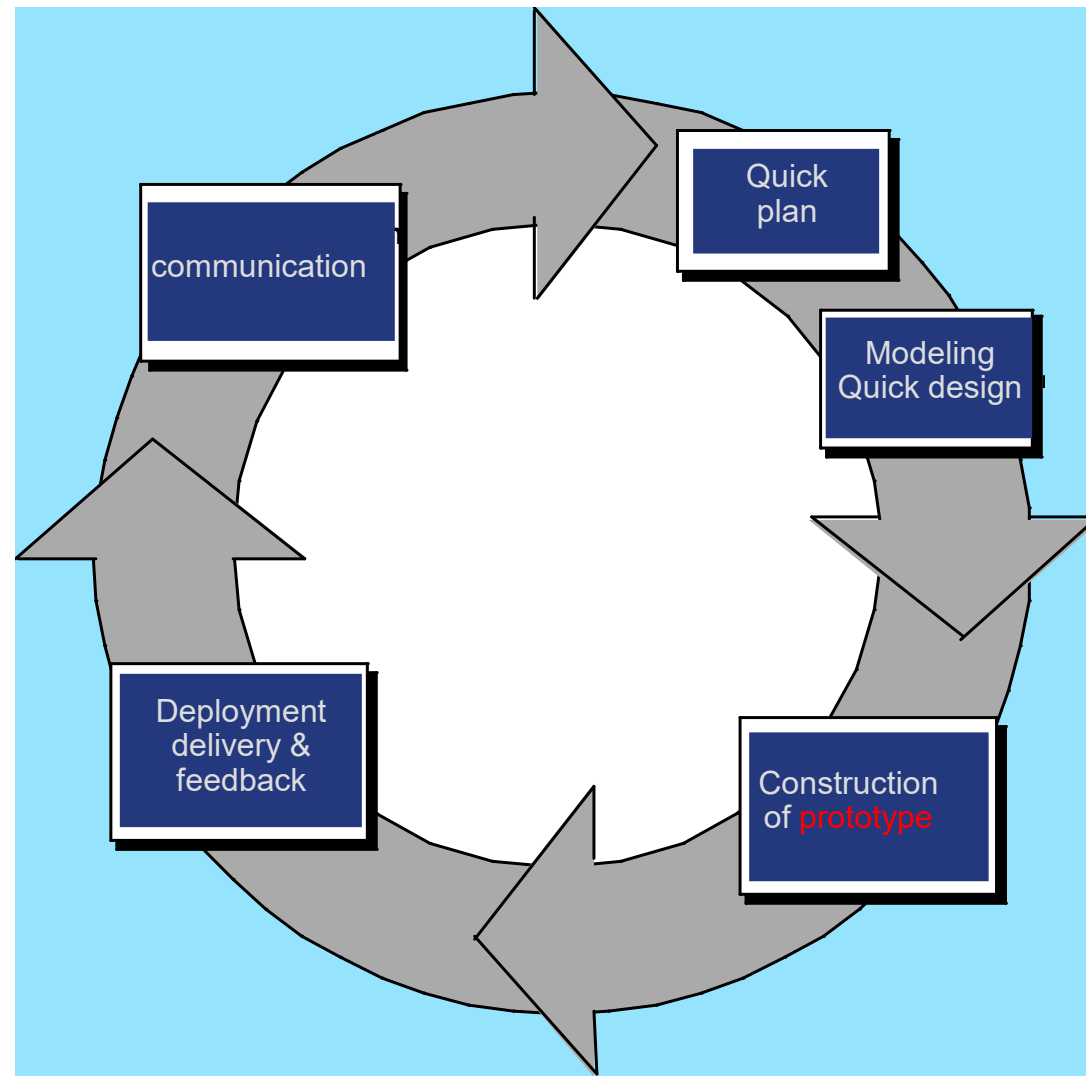
- 需求经常改变
- 时间紧迫
- 只能确定核心需求

3. 演化模型 (原型)

Evolutionary Models: Prototyping



Evolutionary Models: Prototyping



Evolutionary Models: Prototyping

❖ 原型(prototype)的类型:

➤ 探索型(exploratory prototyping)

其目的是要弄清目标系统的要求，确定所希望的特性，并探讨多种方案的可行性

➤ 实验型(experimental prototyping)

其目的是验证方案或算法的合理性，它是在大规模开发和实现前，用于考核方案是否合适，规格说明是否可靠。

➤ 演化型(evolutionary prototyping)

其目的是将原型作为目标系统的一部分，通过对原型的多次改进，逐步将原型演化成最终的目标系统。

Evolutionary Models: Prototyping

❖ 原型(prototype)特征:

- 软件原型是软件的最初版本，以最少的费用、最短的时间开发出的、反映最后软件的主要特征的系统。它具有以下特征：
 - (1)它是一个可实际运行的系统。
 - (2)它没有固定的生存期。它可能被扔掉，或者作为最终产品的一部分。
 - (3)从需求分析到最终产品都可作原型，即可为不同目标作原型。
 - (4)它必须快速、廉价。
 - (5)它是迭代过程的集成部分，即每次经用户评价后修改、运行，不断重复双方认可。

Evolutionary Models: Prototyping

❖ Advantages of the prototyping

- It can help guide **high-quality product requirements**.
- **Relevant data of the project can be obtained at the early stage of the project.** Risk management and configuration management can be carried out as soon as possible.
- It can **encourage** the developers.
- Users can participate in the verification and provide **valuable feedback**.
- It can make the sales work possible to be carried out in advance.

优点

- 一开始就能弄清楚所有的产品需求，或者至少可以帮助导出高质量的产品需求
- 可在项目早期就获取项目的相关数据，尽早进行风险管理和配置管理
- 心理上，开发人员早日见到产品的雏形，是一种鼓舞
- 使用户可以在新的一批功能开发测试后，立即参加验证，以便提供非常有价值的反馈
- 可使销售工作有可能提前进行，因为可以在产品开发的中后期取得包含了主要功能的产品原型去向 各户作展示和试用

Evolutionary Models: Prototyping

❖ Disadvantages of the prototyping

- Without strict process management, prototyping is likely to degenerate into an original unplanned "try-error-fix" cycle mode.
- Psychologically, there may be an idea of “although we can't complete all functions, we have created a product with some functions.”
- If users are exposed to unstable functions without control, it may have a negative impact on developers and users.

■ 缺点

- 如果缺乏严格的过程管理的话，这个生命周期模型很可能退化成为一种原始的无计划的“试——错——改”的循环模式
- 心理上，可能产生一种影响尽最大努力的想法，认为虽然不能完成全部功能，但还是造出了一个有部分功能的产品
- 如果不加控制地让用户接触开发中尚未测试稳定的功能，可能对开发人员及用户都产生负面的影响

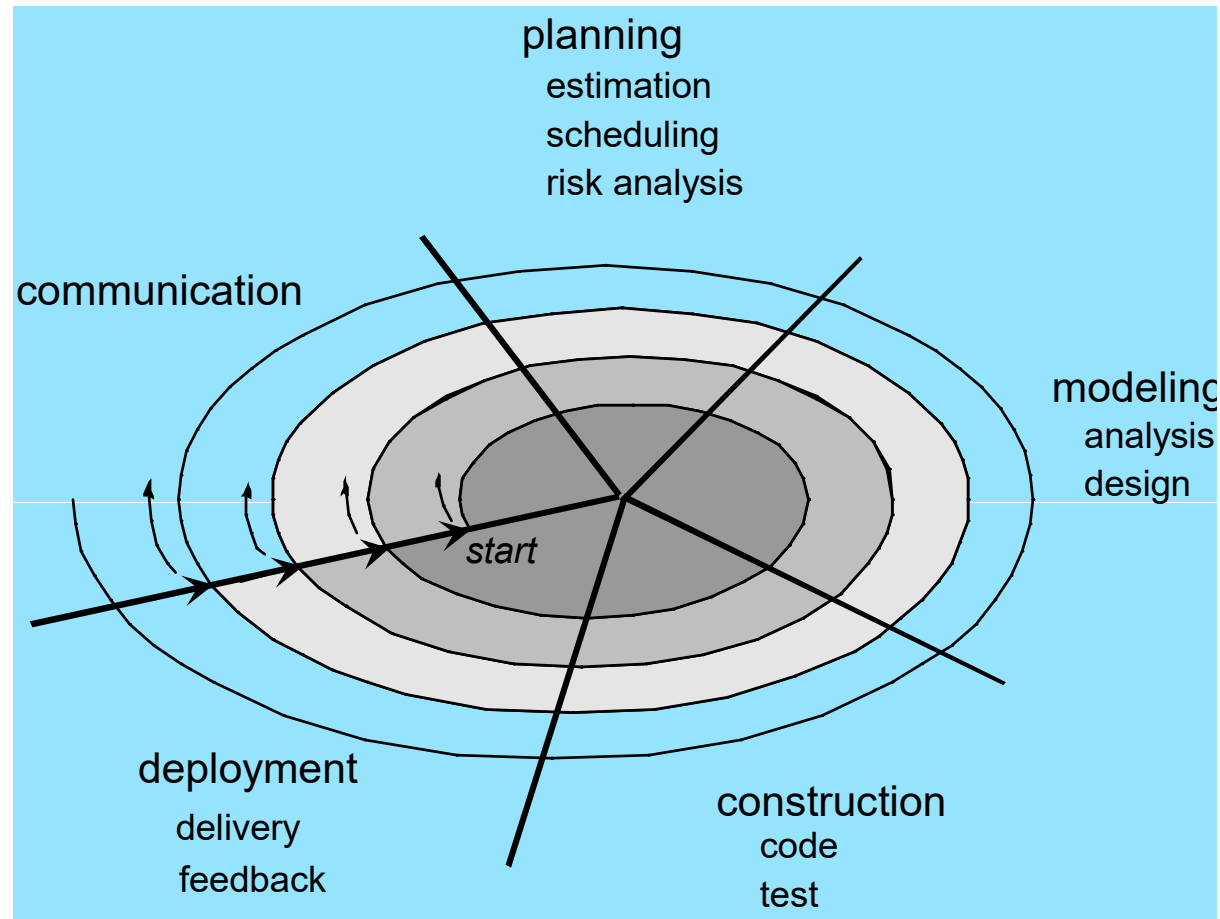
Evolutionary Models: Prototyping

尽管原型制作可以作为一个独立的过程模型使用，但它更常被用作一种技术，可以在本章提到的任何一个过程模型的背景下实施。

❖ Although prototyping can be used as a **standalone process model**, it is more commonly used as **a technique** that can be implemented within the context of any one of the process models noted in this chapter.

4. 演化模型 (螺旋)

Evolutionary Models: Spiral



- ❖ First Proposed: Boehm, B. "Using the WINWIN Spiral Model: A case study", Computer, 31(7), July 1998, PP. 33-44.

Evolutionary Models: Spiral

- ❖ **Using the spiral model, software is developed in a series of evolutionary releases.**
 - **The early release might be a paper model or prototype.**
 - **The latter release can be more complete versions.**
- ❖ **Unlike other process models that end when software is delivered, the spiral model can be adapted to apply throughout the life of the software.**

使用螺旋模型，软件通过一系列演化版本进行开发。

早期版本可能是纸上模型或原型。

后期版本可以是更完整的版本。

不同于其他在软件交付时结束的过程模型，螺旋模型可以适应于整个软件生命周期的应用。

Evolutionary Models: Spiral

- ❖ 在螺旋模型结构中，维护只是螺旋模型的另一个周期，在维护 and 开发之间本质上并没有区别，从而解决了做太多测试或未作足够测试所带来的风险。
- ❖ 优点：
 - a. 强调严格的 **全过程风险管理**。
 - b. 强调各开发阶段的质量。
 - c. 提供机会检讨项目是否有价值继续下去。
- ❖ 缺点：

必须引入非常严格的风险识别，风险分析和风险
控制，这对风险管理的技能水平提出了很高的要
求。也需要人员， **资金和时间的大量投入**。

Evolutionary Models

- ❖ Modern computer software is characterized by **continual change**, by **very tight timelines**, and by an emphatic need for **customer/user satisfaction**.
- ❖ The intent of evolutionary models is to develop high-quality software in an interactive or incremental manner.
However...
- ❖ It is possible to use an evolutionary process to **emphasize flexibility, extensibility, and speed of development**.

现代计算机软件的特点是不断变化、非常紧迫的时间表以及客户/用户满意度的迫切需求。

演化模型的目的是以互动或增量的方式开发高质量的软件。

然而……

可以使用演化过程来强调开发的灵活性、可扩展性和速度。

5. 协调模型 并发模型

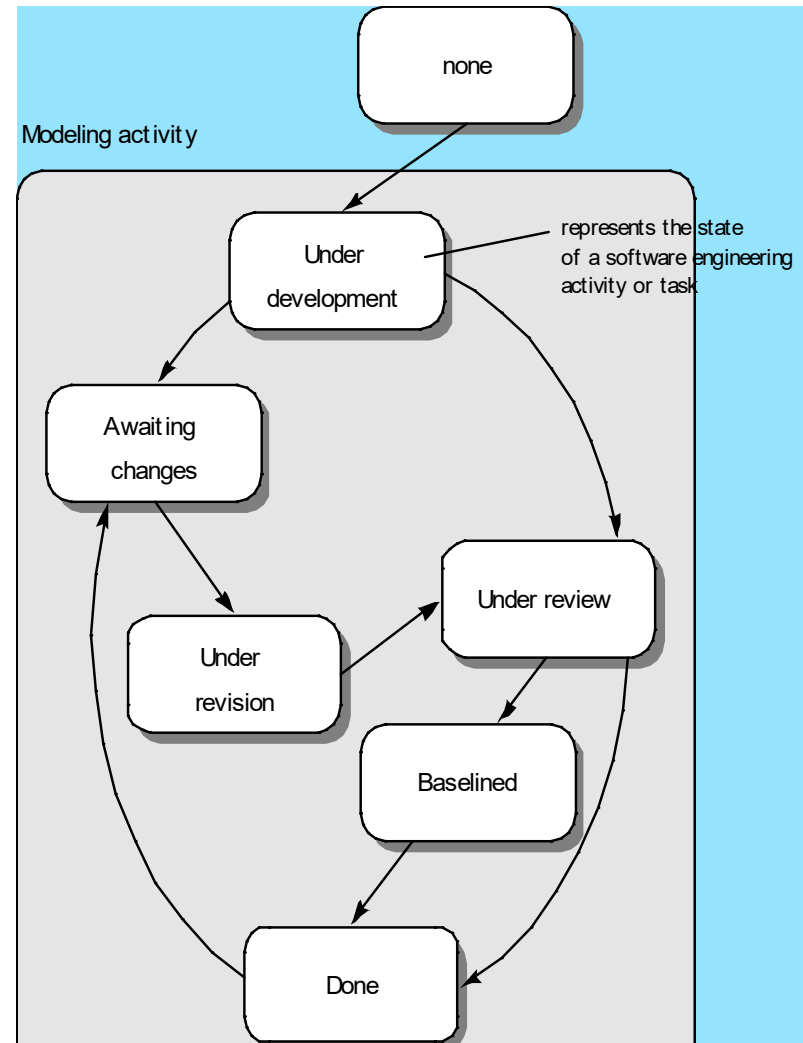
Concurrent Model

❖ Concurrent development model (Concurrent Engineering)

并发开发模型（并发工程）

❖ It defines a series of events that will trigger transitions from state to state for each of the software engineering activities, actions, or tasks.

❖ An example: the concurrent model for modeling activity

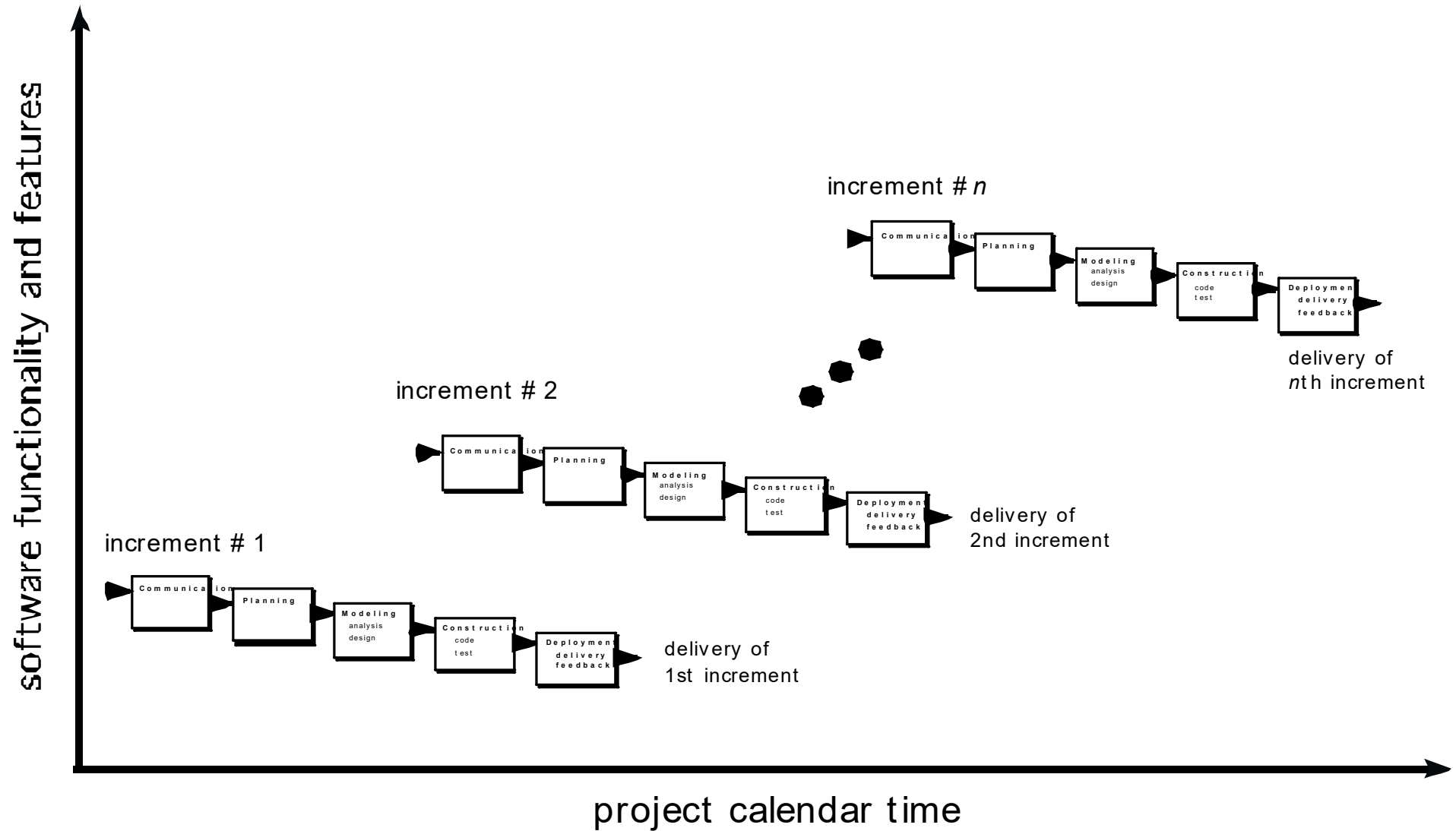


它定义了一系列事件，这些事件将触发从一个状态到另一个状态的过渡，每个软件工程活动、行动或任务都有其相应的状态转换
例如：用于建模活动的并发模型

The Incremental Model

- ❖ 增量模型又称产品改进模型
- ❖ 从给定需求开始，通过构造一系列中间版本来实施开发活动，依次类推，直到系统完成。
- ❖ 每一个中间版本都是需求分析、设计、编码和测试的过程。
- ❖ 某些中间版本的开发可以并行进行。

The Incremental Model



The Incremental Model

❖ 增量模型特征

- 融合了线性顺序模型的基本成分和原型的迭代特征。
- 是随着日程时间的进展而交错的线性序列。
- 与原型不一样的地方是强调每个增量均发布一个可操作产品。
- 最典型的应用是同一个产品的不同项目(合同、用户)版本

The Incremental Model

- ❖ 增量模型特别适用于：
 - 需求经常变化的软件开发
 - 市场急需而开发人员和资金不能在设定的市场期限之前实现一个完善的产品软件开发
- ❖ 增量模型能有计划地管理技术风险，如早期增量版本中避免采用尚未成熟的技术

Other Process Models

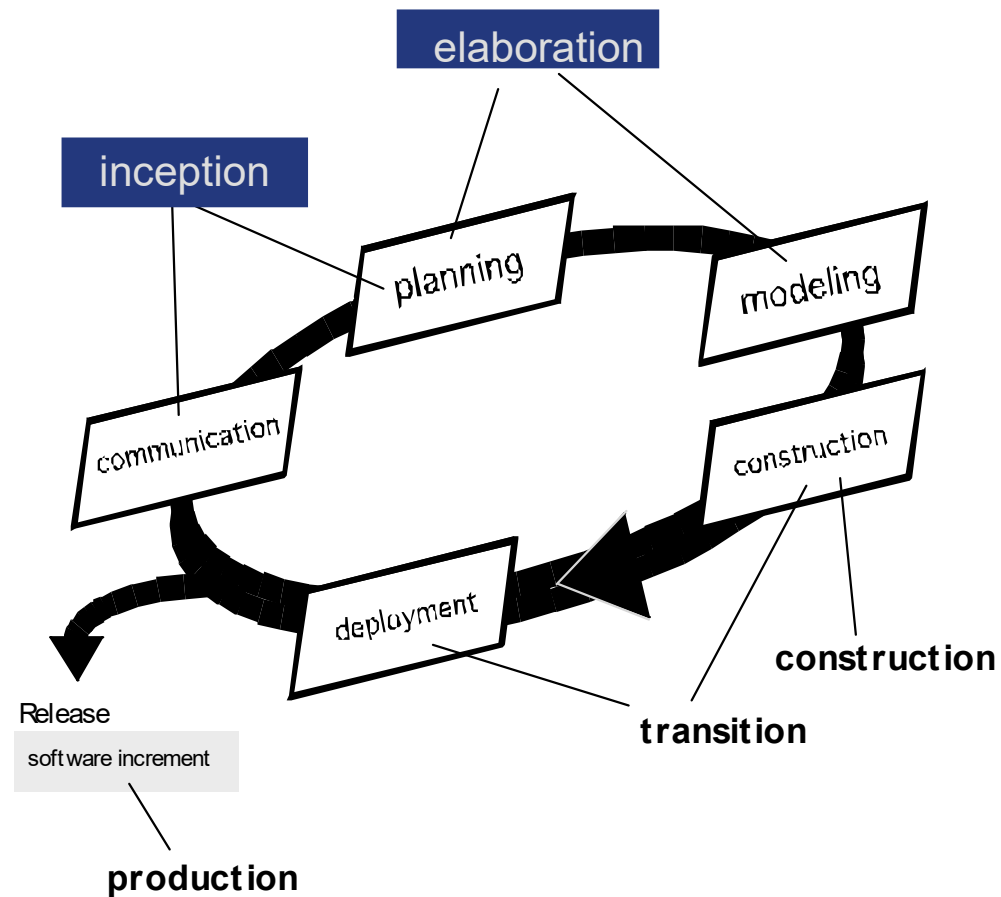
- ❖ **Component based development**—the process to apply when reuse is a development objective
- ❖ **Formal methods**—emphasizes the mathematical specification of requirements
- ❖ **AOSD**—provides a process and methodological approach for defining, specifying, designing, and constructing *aspects*

- 基于组件的开发——当重用是开发目标时应用的过程
- 形式化方法——强调需求的数学规范
- AOSD—提供定义、规范、设计和构建方面的方法和过程

The Unified Process (UP)

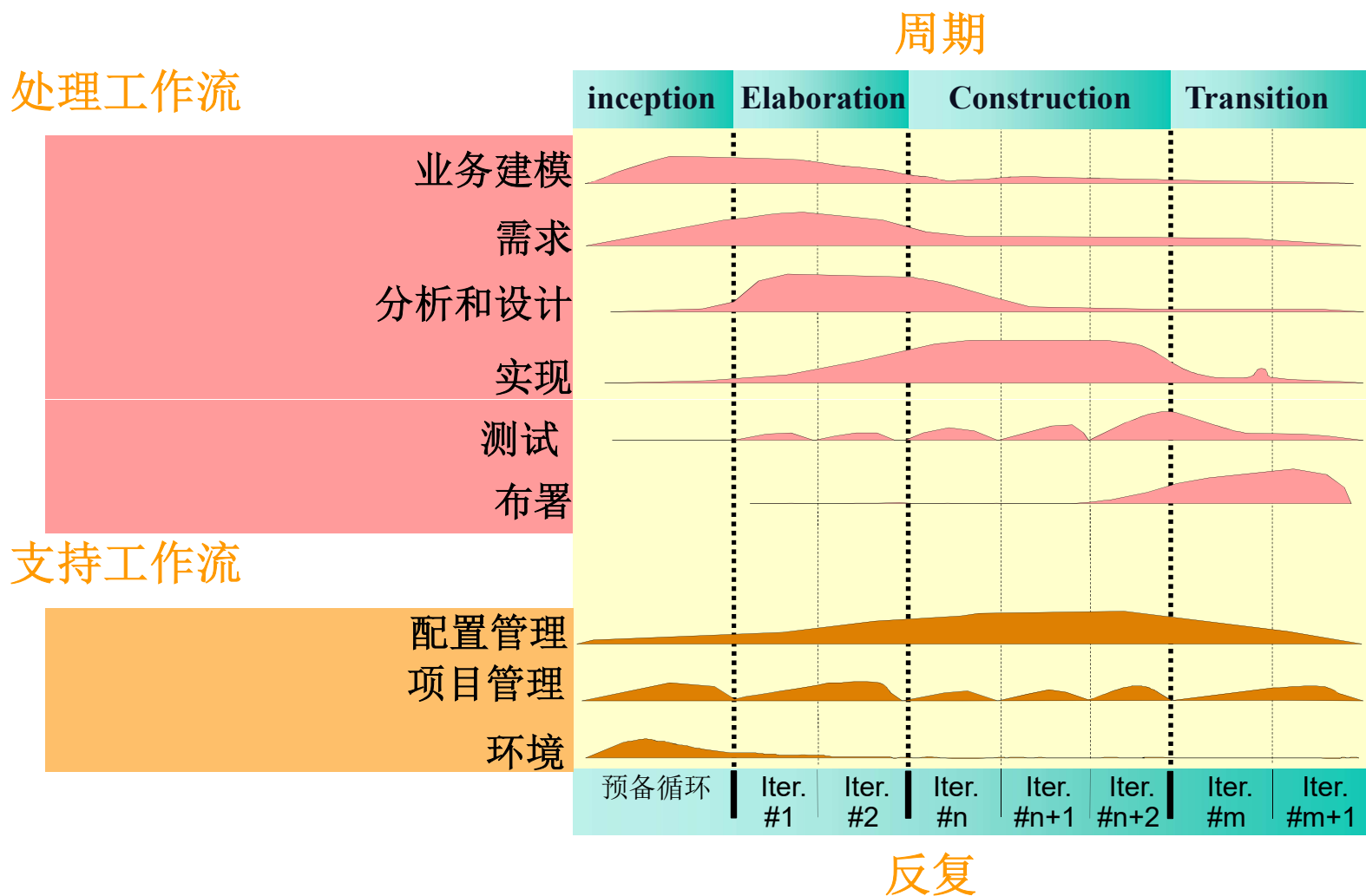
- **RUP (Unified Process)**
 - **I. Jacobson, G. Booch, J. Rumbaugh, 1999**
 - **Object-Oriented**
- **Unified Process**
 - 统一过程 (UP) ☆ ☆
 - 特征:
 - 用例驱动
 - 以架构为核心
 - 迭代并且增量
 - 软件过程与UML紧密结合
 - **use-case driven**
 - **architecture-centric**
 - **iterative and incremental**
 - **software process closely aligned with the Unified Modeling Language (UML)**

The Unified Process (UP)



The Unified Process (UP)

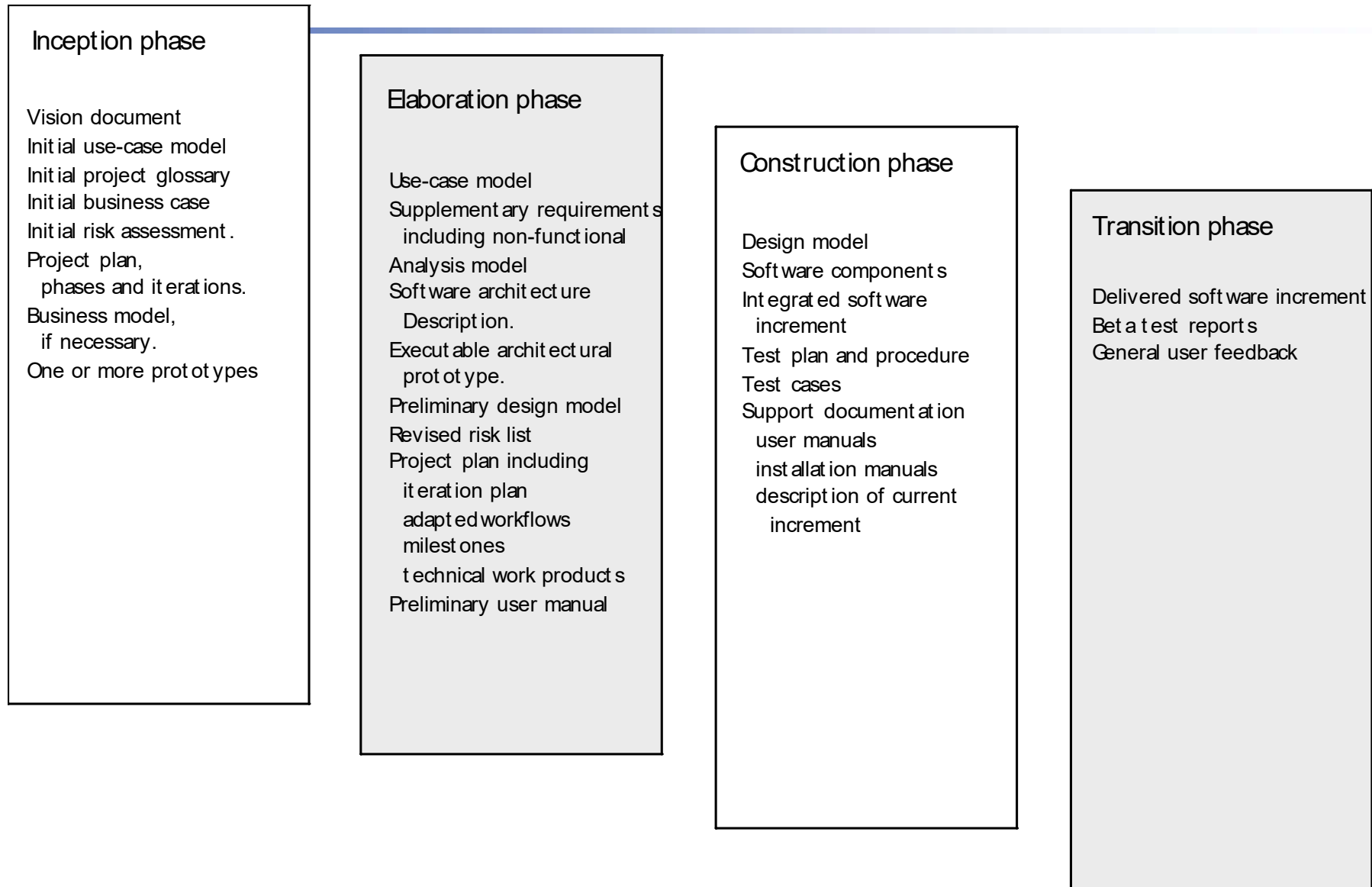
- 时间轴被分成4个阶段：起始、细化、构建、交付



The Unified Process (UP)

- **RUP**的时间轴被分解为四个顺序的阶段：
 - 初始阶段(**Inception**)为系统建立业务案例并确定项目的边界。
 - 细化阶段(**Elaboration**)分析问题领域，建立健全的体系结构基础，编制项目计划，淘汰项目中最高风险的元素。
 - 构造阶段(**Construction**)所有剩余的构件和应用程序功能被开发并集成为产品，所有的功能被详细测试。
 - 交付阶段(**Transition**)确保软件对最终用户是可用的。

UP Work Products



The Unified Process (UP)

- ❖ **RUP的核心 workflow**：分为**6个核心过程 workflow(Core Process Workflows)**和**3个核心支持 workflow(Core Supporting Workflows)**。
- ❖ **核心过程 workflow**
 - **(1)业务建模 workflow**为组织开发一个构想，并基于这个构想在业务用例模型和业务对象模型中定义组织的过程，角色和责任。
 - **(2)需求 workflow**的目标是描述系统应该做什么，并使开发人员和用户就这一描述达成共识。
 - **(3)分析和设计 workflow**将需求转化成未来系统的设计，为系统开发一个健壮的结构并调整设计使其与实现环境相匹配，优化其性能。

The Unified Process (UP)

❖ 核心过程 workflow

- (4)实现 workflow 的目的包括以层次化的子系统形式定义代码的组织结构；以组件的形式(源文件、二进制文件、可执行文件)实现类和对象；将开发出的组件作为单元进行测试以及集成由单个开发者(或小组)所产生的结果，使其成为可执行的系统。
- (5)测试 workflow 要验证对象间的交互作用，验证软件中所有组件的正确集成，检验所有的需求已被正确的实现,识别、提出缺陷并确认缺陷在软件部署之前被处理。
- (6)部署 workflow 的目的是成功的生成版本并将软件分发给最终用户。

The Unified Process (UP)

❖ 核心支持 workflow

- (1)配置和变更管理工作流描绘了如何在多个成员组成的项目中控制大量的产物，管理演化系统中的多个变体，跟踪软件创建过程中的版本。
- (2)软件项目管理 workflow 平衡各种可能产生冲突的目标，管理风险，克服各种约束并成功交付使用户满意的产品。
- (3)环境 workflow 的目的是向软件开发组织提供软件开发环境，包括过程和工具。

The Unified Process (UP)

❖ RUP的优点:

- 提高了团队生产力，在迭代的开发过程、需求管理、基于组件的体系结构、可视化软件建模、验证软件质量及控制软件变更等方面，针对所有关键的开发活动为每个开发成员提供了必要的准则、模板和工具指导，并确保全体成员共享相同的知识基础。它建立了简洁和清晰的过程结构，为开发过程提供较大的通用性。

❖ RUP的缺点

- RUP在理论上，是比较理想的，但在实际应用上，还需要更多的工具的支持和普及推广工作。

Prescriptive Process Model

规范性过程模型提倡一种有序的软件工程方法。

- ❖ Prescriptive process models advocate an orderly approach to software engineering

- ❖ That leads to a few questions ...

If prescriptive process models strive for structure and order, are they **inappropriate** for the **software world that is constantly changing**?

Yet, if we reject traditional process models (and the order they imply) and replace them with **something less structured**, do we make it impossible to achieve coordination and coherence in software work?